

Robotics Security Platform

Interview Preparation — Part 4 of 4: Demo Runbook, Q&A & Cheat-Sheets

Parts 1–3 gave you the knowledge. This part turns it into a performance: how to start the system, the exact live demo to run (and what to say at each step), the CI/CD gate demo, the questions you will almost certainly be asked, the honest trade-offs that earn senior-level credibility, and one-page cheat-sheets to skim minutes before you walk in.

WHY IT MATTERS

Golden rule of demoing: **rehearse the whole thing the morning of the interview.** Never show anything live that you have not run that day. And always have the offline backup (the local lint command in section 3) ready in case the runner misbehaves.

1. How to run the platform

```
# 1. Secrets live in .env (already set). If starting fresh:
cp .env.example .env          # then fill in values

# 2. Build and start the whole stack
docker compose up -d --build

# 3. (No host firewall step.) The router-fw container IS the firewall: it comes up
#     with the stack, enables ip_forward, and loads the default-deny nftables conduits.
#     Verify segmentation any time:
python infra/tests/stage1_connectivity_matrix_docker.py  # expect 16/16 all-green

# 4. Watch the AI container come up (first boot trains the 4 models: 3 network + robot LSTM, ~3-6 min)
docker logs -f container-ai    # wait for "operational"; status should be (healthy)
```

1.2 Where to click (access points)

What	URL	Login
Operations Dashboard (start here)	http://localhost:8888 (or https://localhost:8443)	none (key injected by nginx)
Grafana threat dashboards	http://localhost:3003	admin / GRAFANA_PASSWORD from .env
Prometheus	http://localhost:9090	none
ntopng (network flows)	http://localhost:3001	none
Gitea (CI/CD repo)	http://localhost:3000	set on first run
Guacamole (vendor jump host)	http://localhost:8081	guacadmin / guacadmin
OpenPLC web UI	http://localhost:8080	openplc / openplc

NOTE

After hardening, Modbus (502/503) and the raw API/webhook (8000/9000) are bound to **127.0.0.1** only — reachable from the host for the demo, never from the wider network. The browser UIs above still work normally.

2. The live demo (the money shot, ~5 minutes)

This tells the full story: calm → attack → detect → respond → human-approved stop → deliberate recovery. Run it from the dashboard at localhost:8888.

Step 1 — Show the calm baseline (Overview page)

Point out green zone health, all four models loaded (three network + the robot LSTM), a low flat threat score.

SAY THIS IN THE INTERVIEW

"Everything is green. Four ML models are loaded — three watch the network traffic, a fourth watches the robot's motion — and the robot is cycling normally."

Step 2 — Show the real physical process (PLC Control page)

Show live coils/registers (cycle running, conveyor, gripper) read straight from the PLC over Modbus.

SAY THIS IN THE INTERVIEW

"This is live telemetry polled from the OpenPLC controller — the actual state of the cell."

Step 3 — Launch the attack (AI Engine / Overview)

Trigger the built-in attack injection (a Modbus command-injection / write-burst from an IP outside the OT zone).

SAY THIS IN THE INTERVIEW

"I am injecting a Modbus command-injection — a flood of register writes from outside the OT zone, the classic cyber-physical attack pattern."

Step 4 — Watch detection (Security page)

The threat sparkline spikes within a few seconds; a new anomaly alert appears with the offending source IP and the top contributing features.

SAY THIS IN THE INTERVIEW

"The feature pipeline windowed the traffic, all three models scored it anomalous, and the alert bridge raised it — sub-5-second detection."

Step 5 — Watch automated response (Incidents page)

An incident is open; automatic steps already ran (forensics capture, isolate the attacker IP with a firewall rule). A pending approval waits for the safety step.

SAY THIS IN THE INTERVIEW

"The playbook engine opened an incident and ran the no-approval containment automatically. The step that changes the physical process — asserting a safe state — waits for me."

Step 6 — Approve the safe-state / E-stop

Approve the pending step. On PLC Control you will see e_stop_active set and safety_state = EMERGENCY (latched).

SAY THIS IN THE INTERVIEW

"I approve the safe-state. The E-stop is asserted on the production PLC and the safety system latches EMERGENCY — safety always wins."

Step 7 — Recover deliberately

After "investigating", issue Reset on PLC Control. The cell returns to NORMAL and resumes.

SAY THIS IN THE INTERVIEW

"Recovery is a deliberate operator action, never automatic — correct functional-safety behaviour. Continuity restored."

Step 7b — Show the second detection plane (a robot-behavior attack)

On the AI Engine page, point at the fourth gauge (**Robot LSTM AE**), then inject a robot-plane attack from the grouped dropdown (e.g. **Joint Speed Violation** or **Frozen Joint**).

SAY THIS IN THE INTERVIEW

"The first attack was on the network. But an attacker who already controls the PLC can move the robot with legitimate-looking commands — so a fourth model watches the robot's *motion*. It is a passive tap on the live joint stream, exactly like Zeek on the network."

The Robot LSTM gauge jumps from NOMINAL to ANOMALOUS, a robot-behavior-anomaly alert appears, and it opens its own incident whose containment ladder also ends in a latched E-stop.

SAY THIS IN THE INTERVIEW

"The LSTM autoencoder learned the normal pick-and-place motion, so a tampered trajectory does not reconstruct; the physical-envelope layer names the joint that broke its speed limit. Two layers, one explainable verdict — and the same response loop down to E-stop."

Step 8 (optional) — Show the breadth

- **Stages** page: vulnerability inventory, CVE correlation, baseline drift, integrity, pipeline verdict.
- **Vendor** page: create a time-boxed vendor session and show the Guacamole link plus the audit log.
- **Grafana**: the same telemetry in operational dashboards.

3. The CI/CD gate demo (catch a vulnerability live)

This proves the DevSecOps story: push unsafe PLC code and the build goes **red**; push the safe version and it goes **green**.

```
# RED: introduce the vulnerable program and push -> build fails on Gate 1
cp demos/cicd-gate/demo_jog.VULNERABLE.st vm-ot/openplc/demo_jog.st
git add vm-ot/openplc/demo_jog.st
git commit -m "Add manual jog routine"
git push gitea main          # Gitea Actions tab: red X on "Gate 1 - PLC lint"

# GREEN: ship the hardened version -> build passes
cp demos/cicd-gate/demo_jog.CLEAN.st vm-ot/openplc/demo_jog.st
git add vm-ot/openplc/demo_jog.st
git commit -m "Harden jog routine: E-stop guard, no creds, bounded loop, signed"
git push gitea main          # green check
```

The offline backup (always works, no runner needed)

```
python3 vm-ai/devsecops/plc_lint.py demos/cicd-gate/demo_jog.VULNERABLE.st # exit 1, 7 findings
python3 vm-ai/devsecops/plc_lint.py demos/cicd-gate/demo_jog.CLEAN.st      # exit 0, clean
```

SAY THIS IN THE INTERVIEW

"PLC logic is code, and unsafe code on a robot is a hazard. Every push runs a deterministic gate that blocks unsigned programs, motion paths with no emergency-stop guard, hard-coded credentials, writes to safety outputs from non-safety code, and unbounded loops. Here is a commit it rejected, and the same feature written safely that it accepts — and the Actions step and the in-lab webhook both run the same engine, so a check can never pass in CI yet fail in the plant."

NOTE

The vulnerable file fails with **7 findings**: unsigned program, motion block with no E-stop guard, hard-coded 'admin', two safety-output writes outside a SAFETY_ block, an unbounded FOR loop, and a commented-out safety check. Knowing the exact count and reasons makes the demo look effortless.

4. Likely interview questions (with crisp answers)

Why three ML models on the network plane instead of one?

They are complementary. Isolation Forest cheaply catches rare feature *combinations*; the PCA autoencoder catches windows that do not fit the normal data shape; the TensorFlow autoencoder catches subtler non-linear patterns. If any one crosses its calibrated threshold, we alert — better recall, no single blind spot.

Does anything watch the robot itself, not just the network?

Yes — a second detection plane. A passive tap mirrors the live `/lab_arm/joint_states` stream (just as Zeek mirrors the network) and a fourth model scores the robot's *motion*: an LSTM autoencoder that learned the normal pick-and-place trajectory, plus a deterministic physical-envelope layer (joint speed, range, jerk, frozen-joint). It catches an attacker who already controls the PLC and moves the robot with legitimate-looking commands, and it raises the same kind of incident — ending in a latched E-stop. ROC-AUC is ~0.99 on held-out behavioral attacks.

How do you stop the models drifting from what runs live?

One shared, versioned feature module computes the features for *both* training and live scoring, so the two cannot diverge; the sensors emit only raw values (the joint tap emits raw angles — velocities are derived in that one place). A contract test asserts the live windows are byte-for-byte identical to the training windows, and the feature version is recorded in the model metadata and surfaced by the API.

Why anomaly detection rather than signatures?

OT attacks often "live off the land" using legitimate protocol commands, so there is no signature to match. A model of *normal* flags malicious-but-novel behaviour — for example a register that is normally only read suddenly being written from outside the OT zone.

How do you avoid false positives?

Train on pure normal traffic only; calibrate thresholds to the 99th percentile; score 5-second windows, not single packets; require two consecutive anomalous windows; and apply a per-source cooldown. Multiple detectors tripping on one attack fold into a single incident.

How is the OT zone isolated?

Every service is single-homed on one zone, and a default-deny nftables router is the only node that crosses zones — through eight explicit conduits. IT-to-OT matches no conduit on any path; the analytics zone is network-enforced read-only (it reaches a read-only Modbus proxy, never raw PLC:502); and OT only meets IT through the DMZ. An automated matrix verifies all eight conduits plus key denials (16 checks, all green). That is the IEC-62443 zones-and-conduits model enforced at L3.

What is the single most important control?

The internal OT network plus the latched, watchdog-backed safety system. Segmentation keeps attackers out; the safety system guarantees the robot fails safe regardless of what the attacker or even the AI does.

How fast is detection?

Sub-5-second in the demo — windowed scoring plus the alert bridge.

Where does the AI run — cloud or edge?

At the edge, on-prem, beside the plant, so detection and the safety loop keep working even with no internet. An optional cloud tier only ever receives one-way derived telemetry and sends back signed models/threat-intel; it can never command the plant.

How do you stop the incident-response engine being abused?

Containment that does not touch the physical process runs automatically; anything physical needs operator approval. Command inputs are shell-quoted and IP-validated, and duplicate detections fold into one campaign.

Authentication vs authorization on ROS2?

DDS-Security enforces authentication — every participant must present a CA-signed certificate. Topic-level ACLs are at the permissive default because enforcing the signed permissions hung Cyclone DDS discovery; enforcing them is documented future work.

What protects the safety system specifically?

An independent supervisor with a heartbeat watchdog and a latched E-stop; the safety topics travel over SROS2 with certificate auth; and clearing the safe state is always a deliberate human action.

Why custom Python linters instead of SonarQube / C-STAT / sanitizers?

Those tools scan general application code (mostly C/C++/Java) and cannot read PLC Structured Text, HMI JSON, or SROS2 XML — and they do not know OT safety rules like "motion needs an E-stop guard." They are complementary: in production I would add SonarQube for the dashboard/Python and a certified PLC tool, plugged in as extra gates in the same one-engine pipeline.

What would you improve for production? (the maturity question)

The rearchitecture already closed the big ones — single-homed zones with a default-deny router, network-enforced read-only analytics, and a GPG-signed pull-deploy. What is left (section 5): an independent hardware SIS with local-only reset, per-identity auth + RBAC + mTLS and a secrets vault, HSM-backed signing with SLSA provenance, append-only tamper-evident telemetry, and reconnecting the safety telemetry across the IDMZ.

5. Honest trade-offs (raise these yourself — it wins points)

Interviewers reward candidates who know the limits of their own design. Frame it as: **"This is a faithful teaching/POC build of the full OT-security story; the gap to production is about safety independence, trust boundaries, and honest telemetry — not about the concepts."**

Area	Today (this build)	What I would do for production
Safety controller	Gap: software supervisor co-located in OT (network-unreachable from outside OT, but not independent).	A physically independent Safety Instrumented System (IEC 61511) with a hard-wired, local-only reset and no network un-latch path.
Zoning	Done: single-homed zones + default-deny router (8 conduits); one router is the only multi-homed node.	One host/VLAN per zone (the single host compresses that), optionally a one-way gateway / data diode into analytics.
Analytics tier	Done: network-enforced read-only to OT — reads via a read-only proxy, control via an authenticated OT gateway; no raw PLC write path.	Add mTLS + per-identity authZ on the control gateway; signed control intents.
AuthN/Z	Single shared API key.	Per-identity authentication, RBAC, mTLS between services, signed approvals.
SROS2	Authentication enforced; topic ACLs at permissive default.	Enforce per-topic ACLs once the Cyclone DDS discovery issue is resolved.
CI/CD	Acceptance gate skipped in shipped config; runtime signing key.	Enforce all gates on deploy; sign with an HSM/KMS-backed key; verify provenance (SLSA).
Telemetry	Some synthetic/self-resetting demo telemetry; state cleared on boot.	Append-only, tamper-evident telemetry shipped off-box; preserve audit/forensics.
Secrets	At-rest .env mounted into containers.	A vault/KMS or orchestrator secrets, scoped per service, rotated.

WHY IT MATTERS

Even stronger: the project ran its own security audit, and the rearchitecture then **closed the critical findings** — the monitoring zone no longer has PLC-write authority (now read-only via proxy/gateway), the multi-homed segmentation bypass is gone (single-homed + default-deny router), and deploys are GPG-signed and verified on pull. Being able to say "I audited my own system, fixed the criticals, and here is the honest remaining roadmap" is one of the strongest things you can demonstrate.

6. Deep-dive: segmentation vs microsegmentation (the honest answer)

This is a favourite interview probe, because the requirement brief — and some of our own diagrams — use the word "microsegmentation". The precise, honest answer turns a potential "gotcha" into a credibility win. Read this whole section out loud once; it is written to be spoken.

6.1 The one-paragraph answer to recite

SAY THIS IN THE INTERVIEW

"Today the platform implements an IEC-62443 Level-3.5 IDMZ with **default-deny zone segmentation**: every service is single-homed on one zone, and a single nftables router — the only multi-homed node — permits just eight explicit source→destination:port conduits between zones. That is already a real default-deny *cross-zone* firewall, not just coarse rules. **True microsegmentation** goes one level finer — default-deny between *individual workloads*, including *east-west within* a zone, so each service reaches only the specific peers and ports it needs. To get there I would decompose each broad zone bridge into small per-flow networks (two services that should not talk share no network), and for production move to label-based, default-deny network policies on a CNI like Cilium or Calico with identity-based mTLS between services."

6.2 What we have today vs what microsegmentation adds

What we have (single-homed zones + default-deny cross-zone router):

- Four zone networks (OT .10, IT .20, DMZ .30, MGMT .40); **every container is single-homed** on exactly one, and the router-fw is the only multi-homed node.
- The router runs nftables with a **default-deny forward policy** and exactly **eight allowed conduits** — this is real default-deny *between* zones, verified by a 16-probe matrix.
- **Within a single zone, containers can still reach each other freely** — a Docker bridge allows east-west traffic by default. That intra-zone freedom is the remaining macro→micro gap.
- The analytics zone is already **read-only to OT** at the network layer (proxy for reads, gateway for control), so the most dangerous cross-zone flow is mediated, not open.

What microsegmentation adds: a **default-deny** posture where every individual flow (source → destination → port → protocol) must be explicitly allowed — including *east-west* traffic *within* a zone, not just between zones.

PLAIN-ENGLISH EXAMPLE

Zone segmentation is a building with a few locked floors — once you are on a floor you can walk into any room. Microsegmentation gives **every room its own lock**, and your keycard opens only the rooms your job needs. Default is locked; access is granted one door at a time.

TODAY (zone segmentation)	GOAL (microsegmentation)
+-----+	+-----+
OT ZONE	OT ZONE
PLC <--> safety	PLC --heartbeat--> safety
^ \ / ^ everyone	PLC --:502-----> HMI
\ / in the	sec <--mirror---- (in only)
sec robot zone can	every other flow: DENIED
(reach all peers & ports)	(default-deny east-west)
+-----+	+-----+

6.3 What you would change to implement it

Tier 1 — POC-safe (keeps the demo working, pure Docker Compose)

Replace the few big zone networks with **many tiny purpose-built networks** — one per allowed conversation — and attach each container only to the networks it truly needs. Two containers that share no

network **cannot reach each other**, so default-deny emerges by construction (and it also fixes the multi-homing weakness).

Tiny network	Members	Why it exists
net-span	OT → sec	the read-only traffic mirror
net-hb	production → safety :503	the safety heartbeat only
net-modbus	HMI/score → PLC :502	operator control only
net-redis	sec pusher + ai consumer	the detection bus only
net-scrape	Prometheus → each target	monitoring only
net-ci	gitea + runner + webhook	CI/CD only
net-ops	dashboard → score API	the operator UI only

- Mark every network that does not need the internet **internal: true**.
- Where a network must hold three or more members, disable free chatter with the bridge option **com.docker.network.bridge.enable_icc: "false"**.
- Cross-zone flows are **already** default-deny on the router-fw (nftables, 8 conduits); the new step is to extend that default-deny posture **inside** each zone via the per-flow networks above.
- Single-homing is already done at the zone level (only router-fw is multi-homed); the per-flow networks take it one level finer (per workload).

TRADE-OFF / BE HONEST

Gotcha to mention before they ask: several components **hard-code zone IPs** (for example the Redis host 192.168.40.30, the safety host 192.168.10.11, and the production PLC IP in score_service.py). If you re-cut the networks, switch these to Docker **service-name DNS** (e.g. container-ai) or re-pin them — miss this and the demo breaks silently.

Tier 2 — the production answer (policy- and identity-based)

- Move to Kubernetes with a policy-capable CNI (**Cilium** or **Calico**): write **default-deny NetworkPolicy** plus explicit allow-rules per workload *label*, not per IP. Cilium can even enforce **L7 / Modbus-aware** policy (e.g. allow reads, deny writes).
- **Identity over IP**: give each service a certificate (mTLS / service mesh) so policy follows the workload identity, not a fragile address.
- Add a **one-way gateway / data diode** from OT and SEC into the analytics tier, so the monitoring plane physically cannot send control back.

6.4 If they push: "but your brief / diagram says microsegmentation"

Answer honestly and precisely: "The zone networks are the foundation for microsegmentation, and the brief lists it as a goal. What is enforced today is zone-level; true per-workload microsegmentation is the next step, and I can describe exactly how I would implement it." Then give 6.3. Owning the distinction is far stronger than over-claiming — interviewers test whether you know the difference.

7. Standards, requirements mapping & presentation order

7.1 Standards you are aligned to

Standard	What it covers	Where it shows up here
IEC 62443 / ISA-62443	Industrial control-system security; zones and conduits; SIS security	The whole zone architecture, the DMZ broker, the safety system.
NIST SP 800-82	Guide to OT security	Segmentation, passive monitoring, OT-safe scanning.
NIST SP 800-61	Incident-response lifecycle	The playbook phases: detect, analyse, contain, eradicate, recover, lessons-learned.

7.2 Requirements-to-feature traceability (Topic 114)

Requirement	Delivered by
OT/IT convergence, DMZ, microsegmentation	Single-homed zones + default-deny nftables router (8 conduits, matrix 16/16), analytics read-only to OT, Guacamole DMZ jump host.
Secure remote / vendor access	vendor_access.py + Guacamole, time-boxed audited sessions.
Traffic & protocol monitoring	Zeek + Suricata + ntopng with Modbus/DNP3/OPC-UA parsers.
ML anomaly detection + predictive analytics	Network plane: Isolation Forest + PCA + TF autoencoder over 5s Modbus windows. Robot plane: LSTM autoencoder + physical envelope over joint-motion windows.
Automated response + safety integration	Playbook engine + authenticated E-stop, graded containment.
Safety controls / SIS	Safety supervisor + bridge + heartbeat, latched E-stop, SROS2.
IEC 62443 automation + integrity	Baseline + integrity checks, governance, the CI gates.
Vulnerability management	Nmap inventory, Modbus device-ID, offline CVE correlation, firmware workflow.
DevSecOps for ICS	Six-gate pipeline + GPG-signed pull-deploy (Stage 5): OT verifies the signature before loading; tampered code rejected.
Incident response & recovery + forensics	Playbooks, approvals, forensics capture, continuity-first recovery.

7.3 Suggested presentation order (~6 minutes)

- **1. System architecture** (~90s) — the pieces and how they talk.
- **2. Network segmentation** (~60s) — how they are isolated.
- **3. Detection → response flow** (~90s) — what happens during an attack (run the live demo here).
- **4. Security architecture** (~60s) — the defence-in-depth layers.
- **5. Safety & control loop** (~60s) — how the robot fails safe.
- Then invite questions and lean on sections 4 and 5 above.

8. One-page cheat-sheets

8.1 Key numbers

Thing	Value
Zones / subnets	OT 192.168.10 · IT .20 · DMZ .30 · MGMT .40
Containers	10 (3 custom-built, 7 stock images)
Modbus ports	502 production PLC · 503 safety supervisor
ML models	Network: Isolation Forest · PCA AE · TensorFlow AE. Robot: LSTM AE + physical envelope
Feature window	5-second tumbling, keyed by source IP, 20 features
False-positive brakes	99th-percentile threshold · 2-window debounce · ~45s cooldown
Safety loop	5 Hz heartbeat · ~2-second watchdog · latched EMERGENCY
Detection bus	Redis: features.raw → consumer → anomaly.events → bridge
CI/CD gates	6 (PLC / HMI / SROS2 lint, vuln, baseline, acceptance)
Standards	IEC 62443 · NIST SP 800-82 · NIST SP 800-61

8.2 Mini-glossary (one line each)

Term	Plain meaning
PLC	Rugged industrial computer that directly controls machines.
Structured Text	The IEC 61131-3 language the PLC program is written in.
Modbus	Simple, unauthenticated protocol to read/write PLC registers and coils.
ROS2 / DDS	Modern robot software framework and its messaging transport.
SROS2	The certificate-based security layer for ROS2 messaging.
HMI / SCADA	The operator screen / the wider plant-supervision system.
SIS / E-stop	Independent safety guardian / the emergency-stop function (latched).
Purdue model	The standard way to split a factory network into trust zones.
DMZ	Buffer network where OT and IT meet only through brokered services.
IDS	Intrusion-detection system; here, passive (listens to a mirror only).
Anomaly detection	Learn normal, flag the different — catches novel attacks.
Signed artifact	A build sealed with a hash + signature to prove it is genuine and untampered.
Digital twin	A safe simulated copy (Gazebo) you can attack instead of the real line.

End of Part 4 — and the guide. You now have the knowledge (Parts 1–3) and the performance (Part 4): the pitch, the demo, the answers, and the honest trade-offs. Read Part 1 for the mental model, skim these cheat-sheets right before you walk in, and let the live demo tell the story. Good luck — you have got this.